

Lista de Exercícios de Lista Simplesmente Encadeada – LSE

Avalei e responda as questões:

Com base na estrutura de Lista Simplesmente Encadeada (LSE) de alunos apresentada no material, aqui estão 10 exercícios focados em depuração, lógica de ponteiros e tratamento de casos de borda.

Estes exercícios foram desenhados para que o estudante identifique erros comuns de implementação ou complete lacunas críticas no código.

Exercícios de Lógica e Depuração: LSE

1. O "Nó Órfão" (Alocação)

O código abaixo tenta criar um novo nó, mas contém um erro que impedirá o acesso aos dados do aluno. Identifique o erro:

```
Aluno* novo = (Aluno*) malloc(sizeof(Aluno));
strcpy(novo->nome, "Juca da silva!");
// O que falta aqui para garantir que o nó não aponte para um lixo de memória?
```

2. A Quebra da Corrente (Inserção no Início)

Analise a ordem das instruções. Por que esta função causará a perda de toda a lista original?

```
void insereInicio(LSE *lista, Aluno *novoAluno){
    Aluno* novo = criarAluno(novoAluno);
    lista->primeiro = novo;
    novo->proximo = lista->primeiro;
}
```

3. O Ponteiro Perdido (Busca)

O código abaixo busca um aluno pela matrícula, mas possui um erro de lógica no laço while que pode causar um *Segmentation Fault*. Qual é?

```
void buscaMatricula(LSE *lista, char mat[]){
    Aluno* atual = lista->primeiro;
    while (strcmp(atual->matricula, mat)) {
        atual = atual->proximo;
    }
    return atual;
}
```

4. A Lista Infinita (Inserção no Fim)

Ao inserir no fim, o aluno esqueceu de uma atualização crucial no novo nó. O que acontece ao tentar imprimir essa lista depois?

```
void insereFim(LSE *lista, Aluno *novoAluno){
    Aluno* aux = lista->primeiro;
    while (aux->proximo != NULL) {
        aux = aux->proximo;
    }
    aux->proximo = novoAluno;
    // Falta uma linha de código aqui. Qual?
}
```

5. O Mistério da Lista Vazia (Remoção)

Esta função de remoção no início funciona bem para listas com vários elementos, mas falha miseravelmente em um cenário específico. Qual cenário e por quê?

```
void removeInicio(LSE *lista) {
    Aluno* aux = lista->primeiro;
    lista->primeiro = aux->proximo;
    free(aux);
}
```

6. Salto no Vazio (Contagem)

Para contar os elementos, o aluno escreveu o seguinte código. O resultado da contagem está sempre vindo com um número a menos (ex: tem 5 alunos, mas retorna 4). Corrija a lógica:

```
int contaAlunos(LSE *lista) {
    int cont = 0;
    Aluno* atual = lista->primeiro;
    while (atual->proximo != NULL) {
        cont++;
        atual = atual->proximo;
    }
    return cont;
}
```

7. Ponteiros de Schrödinger (Inserção por Posição)

Para inserir na posição N, é necessário parar no nó N-1. O código abaixo tenta inserir na posição 2 (após o primeiro nó), mas está incompleto. Preencha as lacunas:

```
void inserePosicao(LSE *lista, int N) {
    Aluno* novo = criarAluno("Juca");
    Aluno* anterior = lista->primeiro;
    // Lacuna: Como conectar o 'novo' entre o 'anterior' e o resto da lista?
    _____ = anterior->proximo;
    _____ = novo;
}
```

8. O Vazamento de Memória (Exclusão)

A função apagaLista abaixo limpa o ponteiro de início, mas comete um erro grave de gerenciamento de memória RAM. Explique o que acontece com os nós que estavam na lista:

```
void apagaLista(LSE *lista) {
    lista->primeiro = NULL;
    printf("Lista apagada com sucesso!");
}
```

9. Travessia Cega (Impressão)

Um aluno tentou imprimir os nomes dos alunos, mas o programa entra em loop infinito imprimindo sempre o primeiro nome. O que ele esqueceu dentro do bloco do-while?

```
Aluno *aux = lista->primeiro;
if (aux != NULL) {
    do {
        printf("Nome: %s\n", aux->nome);
        // ???
    } while (aux != NULL);
}
```

10. Ordem Invertida (Lógica Avançada)

Para exibir a turma em ordem invertida (conforme solicitado na tarefa da aula), um aluno sugeriu inverter o ponteiro proximo de cada nó durante a leitura. Quais os riscos dessa abordagem para a estrutura original da lista?

Questionário: Verdadeiro ou Falso (LSE)

1. () Em uma Lista Simplesmente Encadeada, cada nó armazena o dado (carga útil) e o endereço de memória do nó anterior.
2. () O fim de uma LSE é obrigatoriamente identificado quando o ponteiro **proximo** de um nó aponta para **NULL**.
3. () Diferente dos vetores (arrays), a inserção de um elemento no início de uma LSE possui complexidade constante, pois não exige o deslocamento dos demais elementos na memória.
4. () Para acessar o quinto elemento de uma LSE, o computador consegue saltar diretamente para o endereço de memória desse nó, assim como faz em um vetor.
5. () Ao remover o último nó de uma LSE, é necessário percorrer a lista desde o início para encontrar o nó que aponta para ele (o penúltimo), a fim de atualizar seu ponteiro para **NULL**.
6. () A alocação de memória para uma LSE é feita de forma estática, ou seja, o tamanho total da lista deve ser definido no momento da compilação do programa.
7. () O uso da função **free()** é opcional ao excluir um nó de uma LSE em C, pois o sistema operacional identifica automaticamente que aquela memória não está mais sendo usada pelo ponteiro.
8. () Se perdermos o ponteiro que indica o **início** de uma LSE e não houver outro ponteiro auxiliar, todos os dados daquela lista tornam-se inacessíveis, causando um vazamento de memória (*memory leak*).
9. () Em uma LSE, os nós podem estar espalhados em diferentes posições não contíguas na memória RAM.
10. () Uma vantagem da LSE sobre a Lista Duplamente Encadeada (LDE) é o menor consumo de memória por nó, já que a LSE armazena apenas um ponteiro de controle em vez de dois.